AFND com Transições em Vazio
Estados Inacessíveis e Inúteis

Prof(a) Thatyana de Faria Piola Seraphim

Universidade Federal de Itajubá

thatyana@unifei.edu.br

Autômatos Finitos Não-Determinísticos

- Autômatos finitos não-determinísticos que apresentam **transições em vazio** são aqueles que admitem transições de uma estado para outro com ε.
  - Além das transições normais, que utilizam os símbolos do alfabeto de entrada.
- Transições em vazio podem ser executadas sem que seja necessário consultar o símbolo corrente da fita de entrada.
  - E sua execução não causa o deslocamento do cursor de entrada.
- Quando um autômato transita em vazio, significa que ele muda de estado sem consultar a cadeia de entrada.

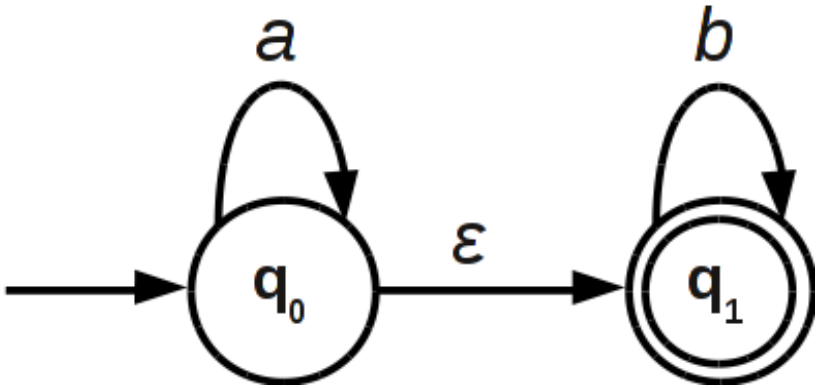
Autômatos Finitos Não-Determinísticos

- Sempre que ocorrer a coexistência entre alguma transição em vazio e outras transições com origem em um mesmo estado
  - isso acarreta a necessidade de uma escolha arbitrária da transição a ser aplicada na configuração, e isso, caracteriza a manifestação de um não-determinismo.
- Seja $M = (Q, \Sigma, \delta, q_0, F)$ um autômato com transições em vazio:

$$
\begin{aligned}
Q &= \{q_0, q_1\} \\
\Sigma &= \{a, b\} \\
\delta &= \{(q_0, a) \rightarrow \{q_0\}, \\
&\qquad (q_0, \varepsilon) \rightarrow \{q_1\}, \\
&\qquad (q_1, b) \rightarrow \{q_1\}\} \\
q_0 &= \{q_0\} \\
F &= \{q_1\}
\end{aligned}
$$

Autômatos Finitos Não-Determinísticos

- A linguagem aceita por esse autômato é a^*b^*, como pode ser visto no diagrama de estados:



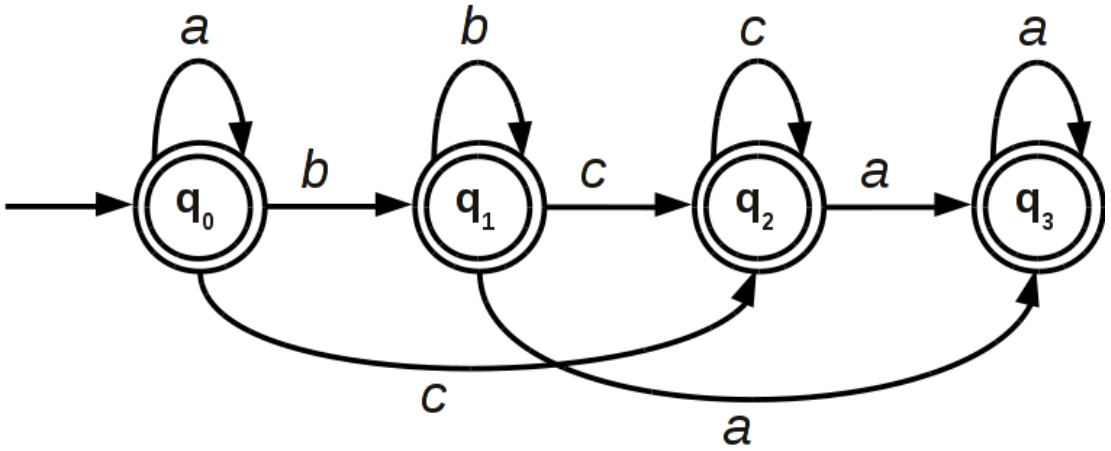
- Dado a cadeia ab, as duas sequências de movimentações possíveis a partir da configuração inicial são:
  1. $(q_0, ab) \vdash (q_0, b) \vdash (q_1, b) \vdash (q_1, \varepsilon)$ (sucesso).
  2. $(q_0, ab) \vdash (q_1, ab)$ (impasse).

Autômatos Finitos Não-Determinísticos

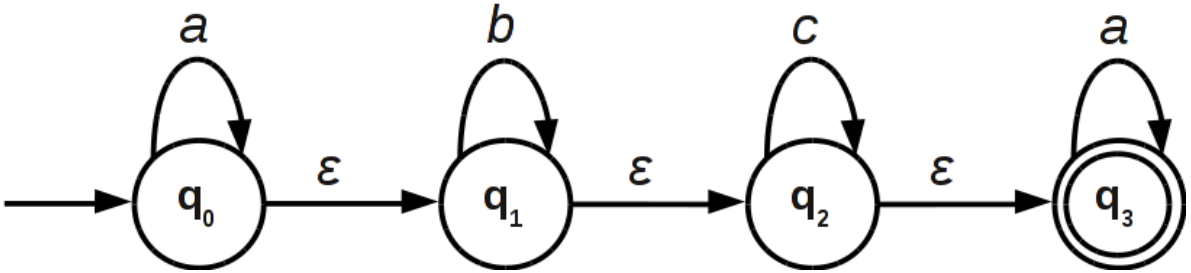
- O impasse ocorre devido à aplicação da transição $(q_0, \varepsilon) \rightarrow q_1$ logo no início do reconhecimento, antes de ser efetuada a leitura do símbolo a.
- No primeiro caso, ocorre a aceitação da sentença, pois a utilização da transição em vazio foi efetuada em um ponto favorável pelo autômato.
  - Entre a utilização dos símbolos a e b da cadeia de entrada.
- Assim como ocorre no caso dos autômatos determinísticos e não-determinísticos, alguns autômatos finitos com transições em vazio se mostram mais simples de serem analisados do que as versões que não possuem transições em vazio.

Autômatos Finitos Não-Determinísticos

- O autômato finito é isento de transições em vazio que reconhece a linguagem $L(M) = a^*b^*c^*a^*$.



- O autômato finito com transições em vazio que reconhece a linguagem $L(M) = a^*b^*c^*a^*$.



Autômatos Finitos Não-Determinísticos

- Apesar de constituir uma generalização da definição dos AFNDs é possível demonstrar que a incorporação de transições em vazio ao autômato nada aumenta o seu poder de reconhecimento.

Teorema (Eliminação de transições em vazio)

Todo autômato com transições em vazio gera uma linguagem que é aceita por algum autômato finito que não contém transições em vazio. Sejam:

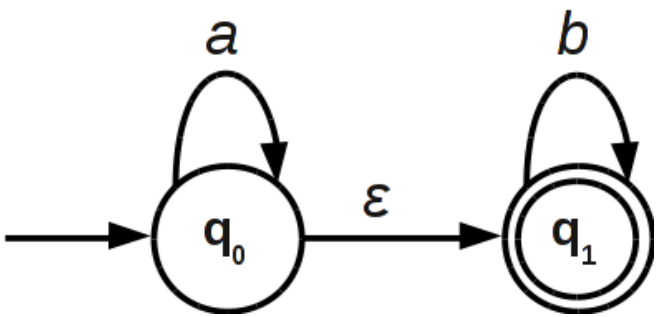
- $M = (Q, \Sigma, \delta, q_0, F)$ *um autômato finito contendo transições em vazio, representado em notação tabular.*
- $N = (Q, \Sigma', \delta, q_0, F')$ *o autômato finito sem transições em vazio correspondente que a partir dele se deseja obter.*
  - *A obtenção de N a partir de M pode ser conseguida através do algoritmo de Eliminação de não-determinismos.*

Autômatos Finitos Não-Determinísticos

- O algoritmo de *eliminação de transições em vazio* funciona através da substituição de transições em vazio por cópias das transições que partem dos estados que seriam atingidos pelas transições em vazio.
- As únicas mudanças do autômato dizem respeito à:
  - **Função de transição** δ': que se torna definida para alguns elementos do domínio de δ indefinidos anteriormente.
  - **Conjunto de estado finais** F': que se amplia se existirem caminhos formados por transições em vazio que interligam estados finais a outros estado do autômato.

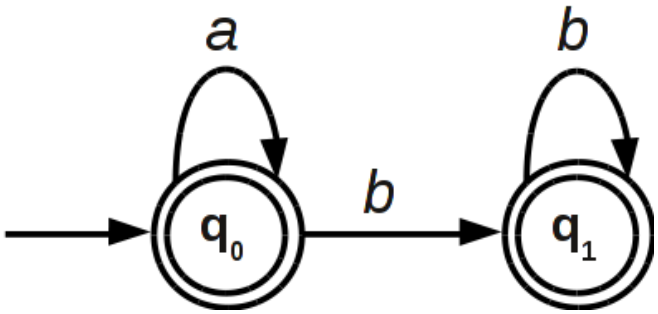
Autômatos Finitos Não-Determinísticos

- Considere o autômato abaixo com a tabela de transição:



	δ	a	b	ε
$\rightarrow$	q_0	q_0		q_1
$\leftarrow$	q_1		q_1	

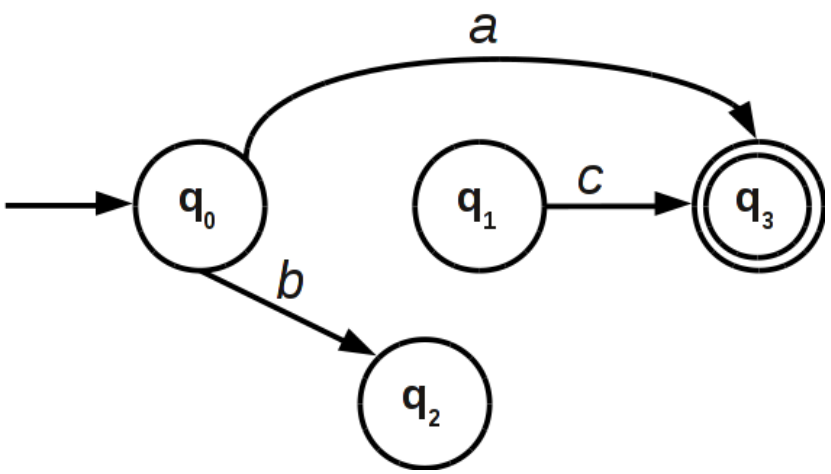
- Como existe uma transição em vazio de q_0 para q_1, deve-se copiar as transições de q_1 para q_0 (apenas neste caso), e considerar q_0 como estado final, uma vez que q_1 é estado final.



	δ'	a	b
$\leftrightarrow$	q_0	q_0	q_1
$\leftarrow$	q_1		q_1

Estados Inacessíveis e Inúteis

- Nem todos os estados de um autômato contribuem para a definição da linguagem por ele aceita.
- **Estados inacessíveis e inúteis** são os mais importantes representantes desta categoria de estados.
  - Os **estados inacessíveis** porque não podem ser alcançados a partir do estado inicial do autômato.
  - Os **estados inúteis** porque não levam a nenhum dos estados finais.



- q_1 é um estado inacessível e q_2 é um estado inútil.

Estados Inacessíveis e Inúteis

- Por se tratar de estados sem relevância, no que se refere à linguagem que se está definindo:
  - é desejável que os mesmos sejam eliminados do autômato, possibilitando sua simplificação.
- **Estados inacessíveis** são aqueles para os quais não existe no autômato qualquer caminho, formado por transições válidas, que permita atingi-los a partir do estado inicial do autômato.
- Os estados inacessíveis podem ocorrer como consequência direta da aplicação de métodos propostos para a eliminação do não-determinismo e/ou da aplicação da remoção de transições em vazio.

Estados Inacessíveis e Inúteis

- Seja $M = \{Q_1, \Sigma, \delta_1, q_{10}, F_1\}$ um autômato finito qualquer.
  - Um estado $q_{1i} \in Q_1$ é dito **inacessível** quando não existir caminho, formado por transições válidas que conduzam o autômato do seu estado inicial q_{10} até o seu estado final q_{1i}.
  - Não existe $\alpha \in \Sigma^*$ tal que $\delta(q_{10}, \alpha) = q_{1i}$. Caso contrário, o estado q_{1i} é dito **acessível**.
- Estado inacessíveis não contribuem para a definição da linguagem aceita por M,
  - podendo ser identificados e eliminados do conjunto de estados, sem prejuízo para a linguagem aceita pelo autômato.

Estados Inacessíveis e Inúteis

Algoritmo (Eliminação de estados inacessíveis)

Obtenção de um autômato sem estados inacessíveis equivalente a outro com estados inacessíveis.

- **Entrada**: um autômato finito $M = (Q_1, \Sigma, \delta_1, q_{10}, F_1)$.
- **Saída**: um autômato finito $N = (Q_2, \Sigma, \delta_2, q_{20}, F_2)$, isento de estados inacessíveis, e tal que $L(N) = L(M)$.
- **Método**:
  1. $Q_{20} \leftarrow \{q_{20}\}$.
  2. $i \leftarrow 1$.
  3. $Q_{21} \leftarrow Q_{2(i-1)} \cup \{q_{2k} | q_{1k} \in Q_1$ e $\delta_1(p, \sigma) = q_{1k}, p \in Q_{2(i-1)}, \sigma \in (\Sigma \cup \{\varepsilon\})\}$.
  4. Se $Q_{2i} \neq Q_{2(i-1)}$, então $i \leftarrow i+1$ e desviar para (3); caso contrário, $Q_2 \leftarrow Q_{2i}$.
  5. $F_2 \leftarrow \{q_{2k} \in Q_2 | q_{1k} \in F_1\}$.
  6. $\delta_2 \leftarrow \{(q_{2i}, \sigma) \rightarrow q_{1j} \in \delta_1$ e $q_{2i}, q_{2j} \in Q_2\}$.

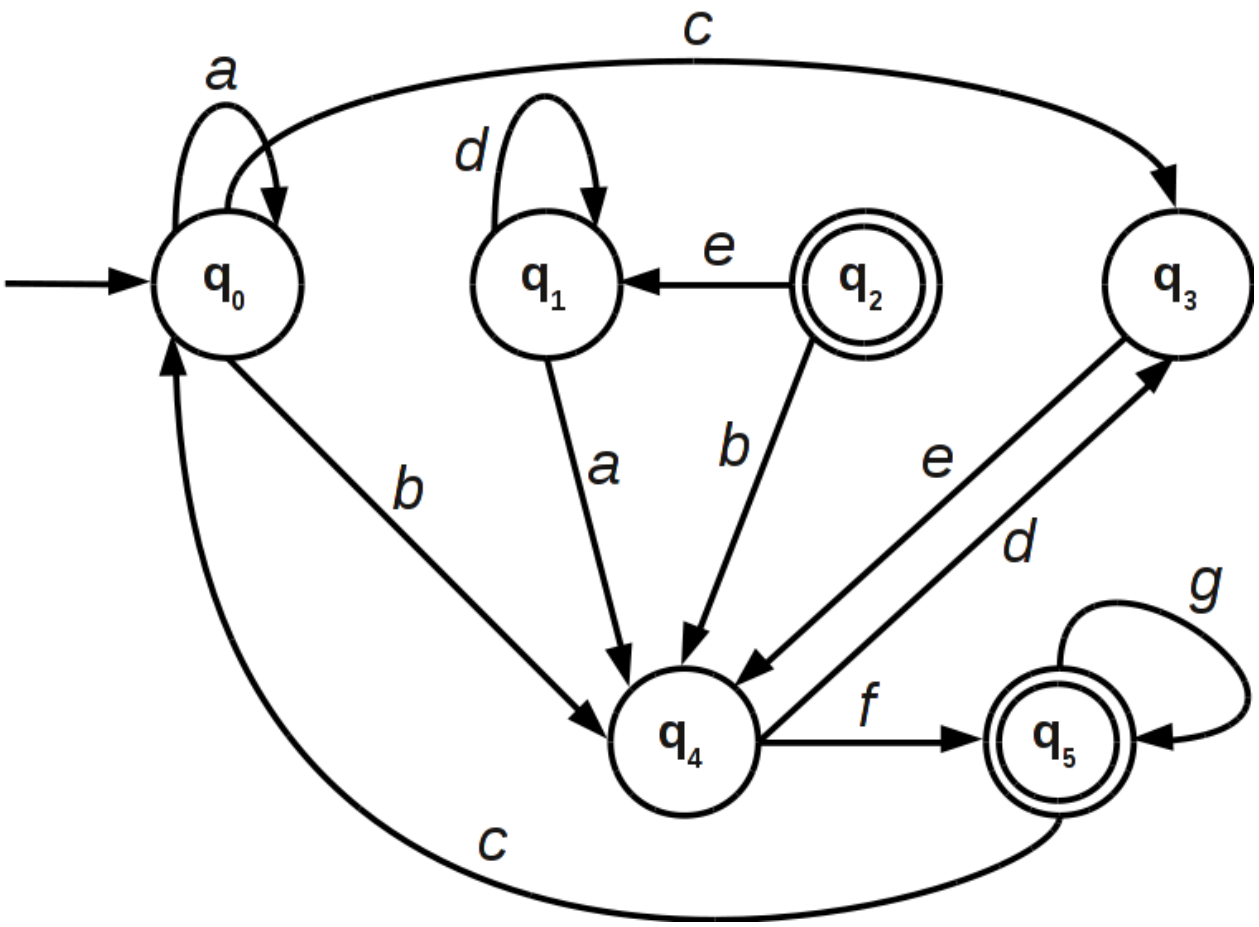
Estados Inacessíveis e Inúteis

- A função de transição δ do autômato é representada na forma tabular, com duas colunas a mais na tabela:
  - **Acessível**: com a função de marcar os estados acessíveis.
  - **Considerado**: com a função de indicar que o correspondente estado já foi levado em conta pelo método do algoritmo.
- Por exemplo: seja $M = (Q, \Sigma, \delta, q_0, F)$ representado abaixo.

	δ	a	b	c	d	e	f	g
$\rightarrow$	q_0	q_0	q_4	q_3				
	q_1	q_4			q_1			
$\leftarrow$	q_2		q_4			q_1		
	q_3					q_4		
	q_4				q_3		q_5	
$\leftarrow$	q_5			q_0				q_5

Estados Inacessíveis e Inúteis

- Por exemplo: seja $M = (Q, \Sigma, \delta, q_0, F)$ representado abaixo.



Estados Inacessíveis e Inúteis

Iniciando o procedimento:

- Marca q_0 como acessível

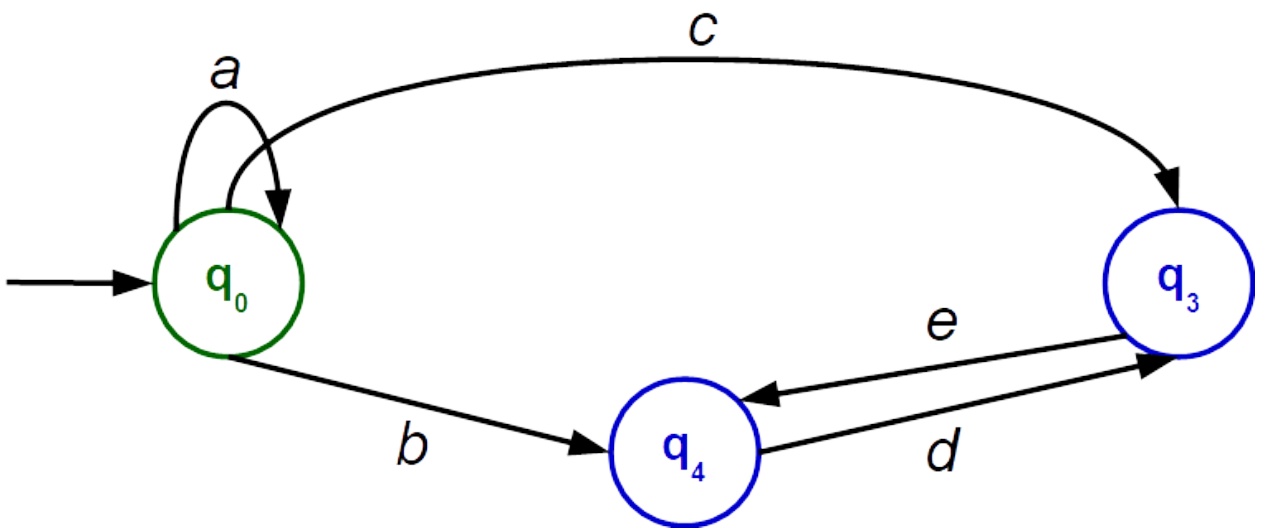
	δ	a	b	c	d	e	f	g	*Acessível*	*Considerado*
$\rightarrow$	q_0	q_0	q_4	q_3					✓	
	q_1	q_4			q_1					
$\leftarrow$	q_2		q_4			q_1				
	q_3					q_4				
	q_4				q_3		q_5			
$\leftarrow$	q_5			q_0				q_5		

Tratando o (único) estado acessível ainda não considerado q_0 (que referencia q_0, q_4, q_5)

- q_0 já estava marcado como **acessível**.
- q_4 é marcado como **acessível**.
- q_3 é marcado como **acessível**.
- Ao final, q_0 é marcado como **considerado**.

Estados Inacessíveis e Inúteis

	δ	a	b	c	d	e	f	g	*Acessível*	*Considerado*
$\rightarrow$	q_0	q_0	q_4	q_3					✓	✓
	q_1	q_4			q_1					
$\leftarrow$	q_2		q_4			q_1				
	q_3					q_4			✓	
	q_4				q_3		q_5		✓	
$\leftarrow$	q_5			q_0				q_5		



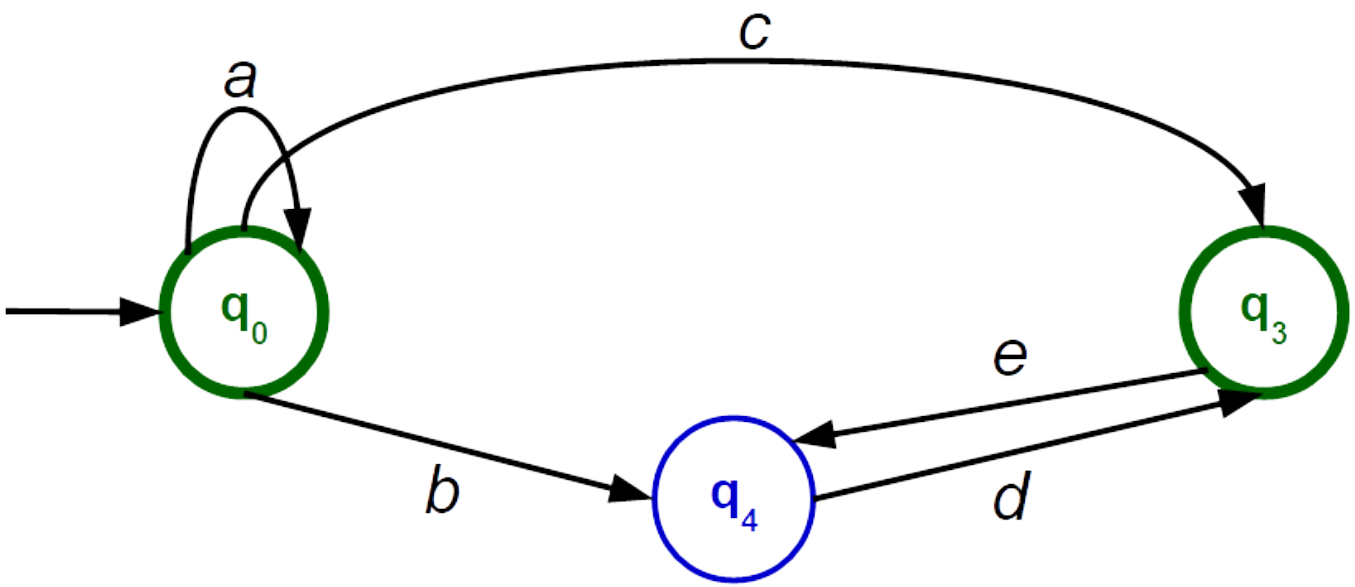
Estados Inacessíveis e Inúteis

Tratando o estado acessível não-considerado q_3 (que referencia q_4).

- q_4 já estava marcado como **acessível**.
- Ao final q_3 é marcado como **considerado**.

	δ	a	b	c	d	e	f	g	*Acessível*	*Considerado*
$\rightarrow$	q_0	q_0	q_4	q_3					✓	✓
	q_1	q_4			q_1					
$\leftarrow$	q_2		q_4			q_1				
	q_3					q_4			✓	✓
	q_4				q_3		q_5		✓	
$\leftarrow$	q_5			q_0				q_5		

Estados Inacessíveis e Inúteis



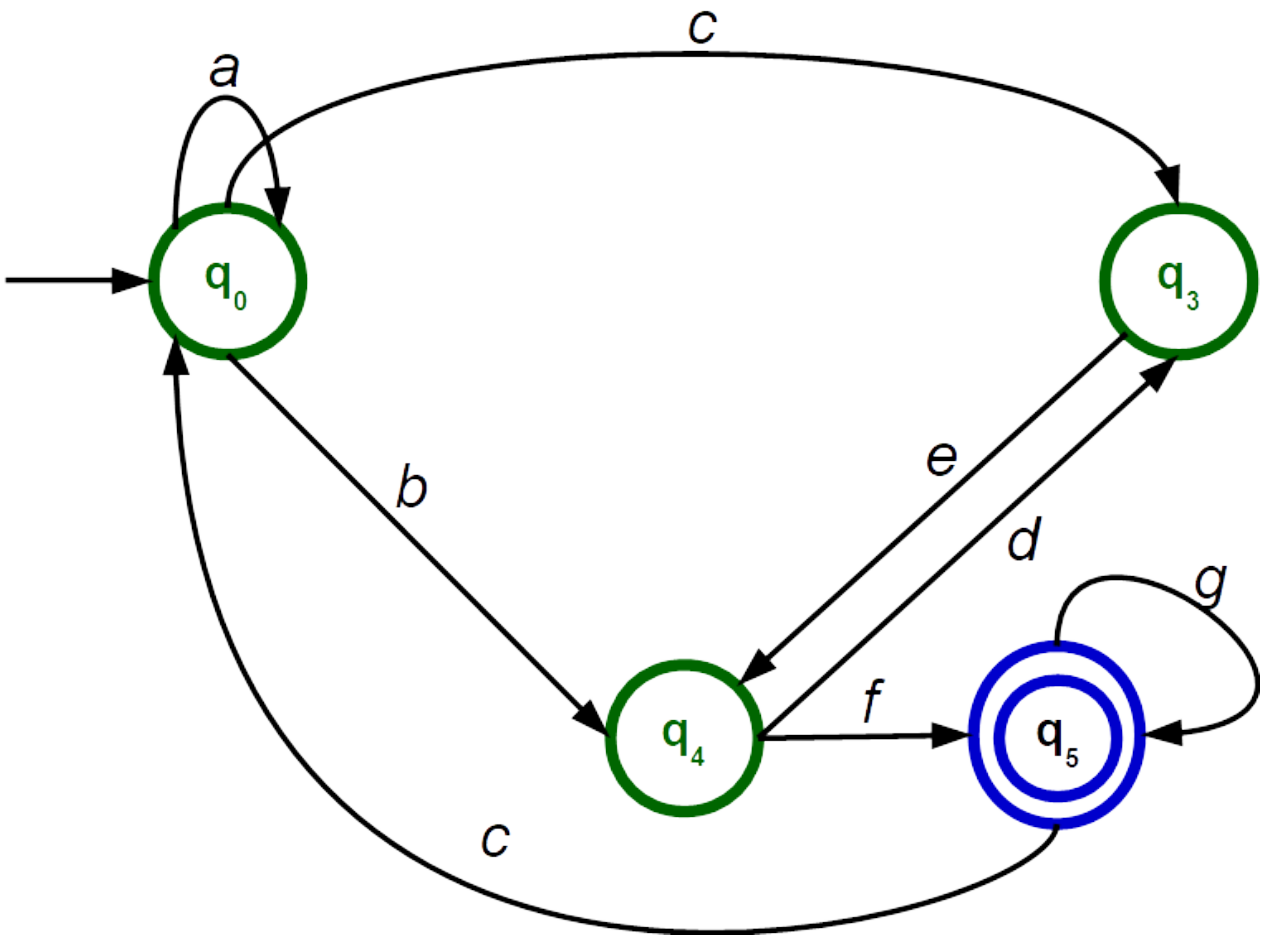
Estados Inacessíveis e Inúteis

Tratando o (único) estado acessível não-considerado q_4 (que referencia q_3 e q_5)

- q_3 já estava marcado como **considerado**.
- q_5 é marcado como **acessível**.
- Ao final q_4 **considerado**.

	δ	a	b	c	d	e	f	g	*Acessível*	*Considerado*
$\rightarrow$	q_0	q_0	q_4	q_3					✓	✓
	q_1	q_4			q_1					
$\leftarrow$	q_2		q_4			q_1				
	q_3					q_4			✓	✓
	q_4				q_3		q_5		✓	✓
$\leftarrow$	q_5			q_0				q_5	✓	

Estados Inacessíveis e Inúteis



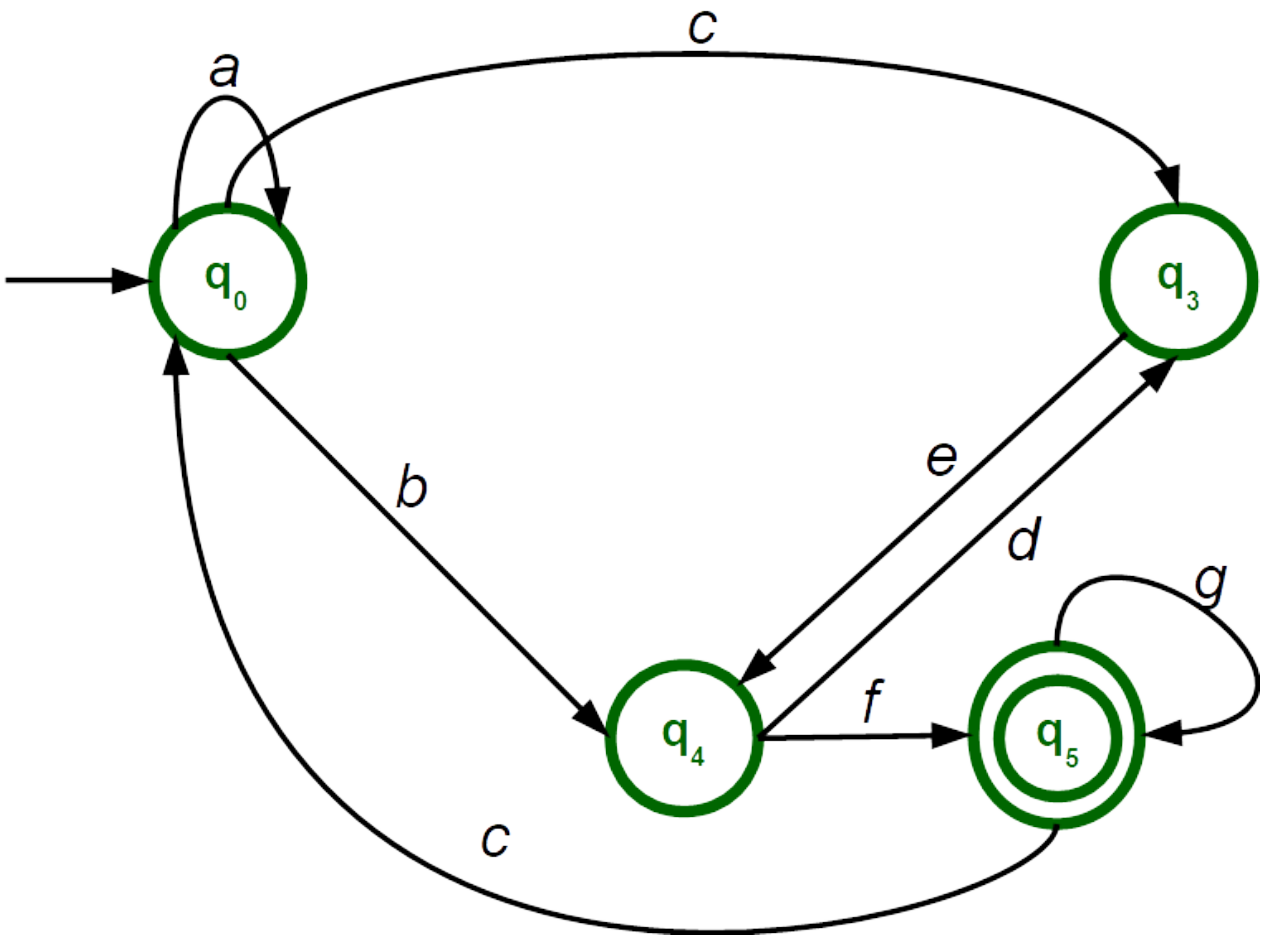
Estados Inacessíveis e Inúteis

Tratando o (único) estado acessível não-considerado q_5 (que referencia q_0 e q_5)

- q_0 já estava marcado como **acessível**.
- q_5 também já estava marcado como **acessível**.
- Ao final, q_5 é marcado como **considerado**.

	δ	a	b	c	d	e	f	g	*Acessível*	*Considerado*
$\rightarrow$	q_0	q_0	q_4	q_3					✓	✓
	q_1	q_4			q_1					
$\leftarrow$	q_2		q_4			q_1				
	q_3					q_4			✓	✓
	q_4				q_3		q_5		✓	✓
$\leftarrow$	q_5			q_0				q_5	✓	✓

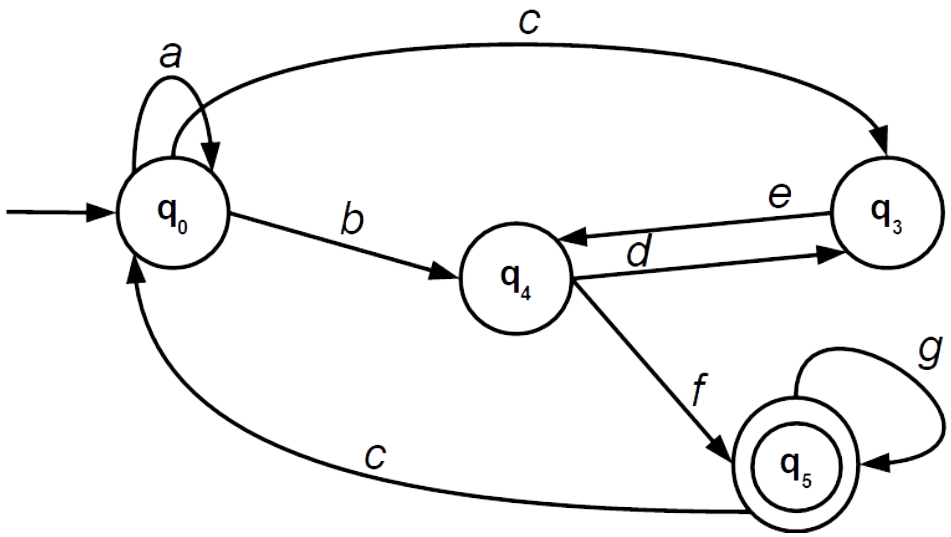
Estados Inacessíveis e Inúteis



Estados Inacessíveis e Inúteis

- Nada mais havendo para fazer, o algoritmo se encerra descartando as duas colunas auxiliares e as linhas correspondentes aos estados q_1 e q_2, que permanecem sem a marca **acessível** até o final.

	δ	a	b	c	d	e	f	g
$\rightarrow$	q_0	q_0	q_4	q_3				
	q_3					q_4		
	q_4				q_3		q_5	
$\leftarrow$	q_5			q_0				q_5



Estados Inacessíveis e Inúteis

- **Estados inúteis** são estados que, apesar de poderem ser alcançados a partir do estado inicial do autômato, não conduzem a nenhum dos seus estados finais.
  - Eles nada contribuem para a aceitação de sentenças das linguagens definidas pelo autômato, podendo ser removidos sem qualquer prejuízo para a linguagem reconhecida.
- Os estados inúteis podem ser identificados e eliminados usando o algoritmo **Eliminação de não-determinismos**.
- A base usada no algoritmo de eliminação de estados inúteis será o conjunto dos estados finais do autômato
  - que são sempre úteis (todo estado final reconhece pelo menos uma cadeia, a cadeia vazia).
  - Se não houver estados finais no autômato, todos os seus estados podem ser declarados inúteis e a linguagem por ele definida será vazia.

Estados Inacessíveis e Inúteis

- Seja o autômato original $M = (Q_1, \Sigma, \delta_1, q_{10}, F_1)$ e $N = (Q_2, \Sigma, \delta_2, q_{20}, F_2)$ o autômato que se deseja obter.
- O algoritmo de eliminação de não-determinismos permite a obtenção de N a partir de M.

Algoritmo (Eliminação de estados inúteis)

Obtenção de um autômato sem estados inúteis equivalente a outro com estados inúteis, porém sem estados inacessíveis.

- **Entrada**: um autômato finito $M = (Q_1, \Sigma, \delta_1, q_{10}, F_1)$.
- **Saída**: um autômato finito $N = (Q_2, \Sigma, \delta_2, q_{20}, F_2)$, isento de estados inúteis, e tal que $L(M) = L(N)$.

Estados Inacessíveis e Inúteis

- **Método**:
  1. $Q_{20} \leftarrow \{q_{2i} | q_{1i} \in F_1\}$
  2. $i \leftarrow 1$
  3. $Q_{2i} \leftarrow Q_{2(i-1)} \in \{q_{2k} | q_{1k} \in Q_1$ e $\delta_1(q_{1k}, \sigma) = q, q \in Q_{i-1}, \sigma \in (\Sigma \in \{\varepsilon\})\}$
  4. Se $Q_2 \neq Q_{2(i-1)}$, então $i \leftarrow i+1$ e desviar para (3); caso contrário, $Q_2 \leftarrow Q_{2i}$
  5. $F_2 \leftarrow \{q_{2k} \in Q_2 | q_{1k} \in F_1\}$
  6. $\delta_2 \leftarrow \{(q_{2i}, \sigma) \rightarrow q_{2j} | (q_{1i}, \sigma) \rightarrow q_{1j} \in \delta_1$ e $q_{2i}, q_{2j} \in Q_2\}$
- A função de transição δ pode ser representada na forma tabular acrescentando duas novas colunas à tabela: a primeira denominada **Útil** e a segunda chamada **Considerado**, e aplicar os passos do algoritmo de eliminação de não-determinismos.

Estados Inacessíveis e Inúteis

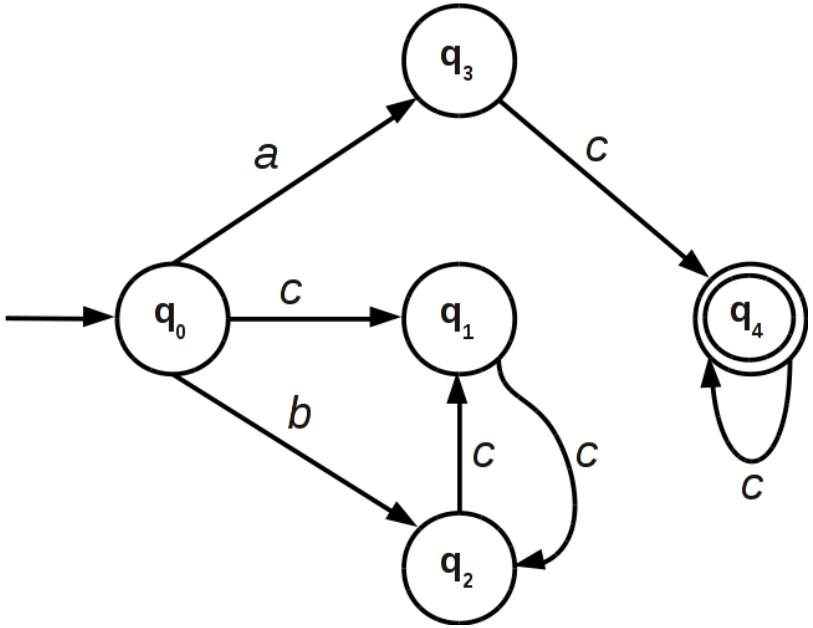
Algoritmo (Eliminação de estados inúteis 2)

Método prático para obtenção de um autômato sem estados inúteis equivalente a outro com estados inúteis, porém sem estados inacessíveis.

- **Entrada**: um autômato finito $M = (Q_1, \Sigma, \delta_1, q_{10}, F_1)$.
- **Saída**: um autômato finito $N = (Q_2, \Sigma, \delta_2, q_{20}, F_2)$, isento de estados inúteis, e tal que $L(M) = L(N)$.
- **Método**:
  1. Considera-se as linhas correspondentes aos estados finais de M. Marca-se as mesmas como úteis, na coluna apropriada.
  2. Seleciona um estado qualquer marcado como **útil**, porém ainda não marcado como **considerado**.
     - Verifica todos os demais estados identificando quais permitem transitar para o estado selecionado.
     - Marca todos como **útil**.
     - Marca finalmente o estado selecionado como **considerado**.

Estados Inacessíveis e Inúteis

- **Método**:
  3. Repita o passo (2) até que não reste mais nenhum estado marcado como **útil**, mas ainda não como **considerado**.
  4. O autômato N é definido a partir de M eliminando todos os estado inúteis, bem como todas as transições que deles partem ou a eles chegam.
- **Exemplo** Considere o autômato da figura, em que todos os estados são acessíveis mas nem todos são úteis.

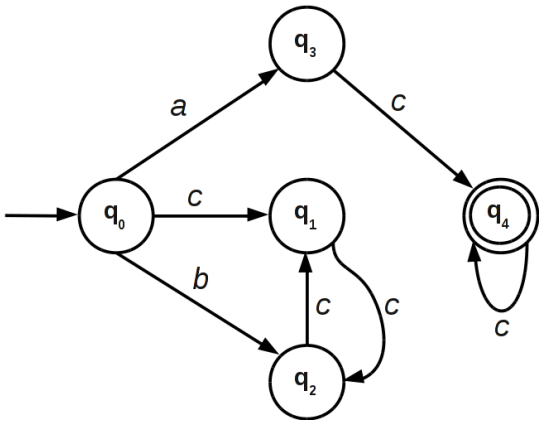


Estados Inacessíveis e Inúteis

- Inicia-se estendendo a tabela de transição com as colunas **útil** e **considerado**, e marca como úteis todos os estados finais.
- Marca como **útil** o estado q_4.

	δ	a	b	c	*Útil*	*Considerado*
$\rightarrow$	q_0	q_3	q_2	q_1		
	q_1			q_2		
	q_2			q_1		
	q_3			q_4		
$\leftarrow$	q_4			q_4	✓	

Estados Inacessíveis e Inúteis



- As duas únicas referências a q_4 estão nos estados q_3 e q_4.
- Marca q_3 como **útil**.
- q_4 já estava marcado.
- Ao final, marca q_4 como **considerado**.

	δ	a	b	c	*Útil*	*Considerado*
$\rightarrow$	q_0	q_3	q_2	q_1		
	q_1			q_2		
	q_2			q_1		
	q_3			q_4	✓	
$\leftarrow$	q_4			q_4	✓	✓

Estados Inacessíveis e Inúteis

Seleciona em seguida q_3, o único estado útil não considerado:

- A única referência a q_3 está no estado q_0.
- Marca q_0 como **útil**.
- Ao final, marca q_3 como **considerado**.

	δ	a	b	c	*Útil*	*Considerado*
$\rightarrow$	q_0	q_3	q_2	q_1	✓	
	q_1			q_2		
	q_2			q_1		
	q_3			q_4	✓	✓
$\leftarrow$	q_4			q_4	✓	✓

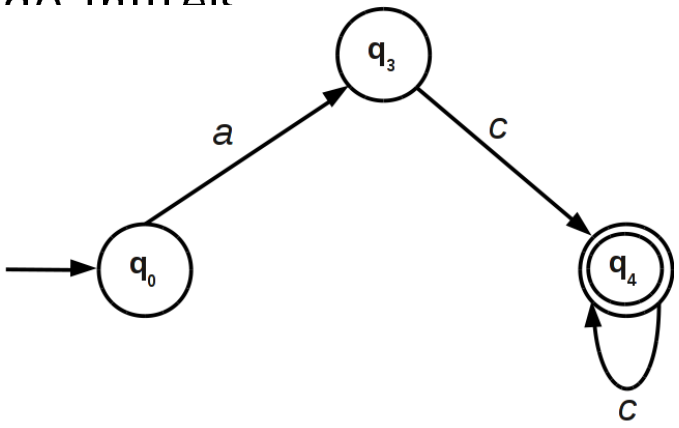
Estados Inacessíveis e Inúteis

- Seleciona q_0, o único estado útil não **considerado**.
- Como q_0 não é referenciado por nenhum outro estado, então ele é marcado como **considerado**.

	δ	a	b	c	*Útil*	*Considerado*
$\rightarrow$	q_0	q_3	q_2	q_1	✓	✓
	q_1			q_2		
	q_2			q_1		
	q_3			q_4	✓	✓
$\leftarrow$	q_4			q_4	✓	✓

- Não existindo mais estados úteis não considerados, o algoritmo se encerra: q_1 e q_2 são estado inúteis

	δ	a	b	c
$\rightarrow$	q_0	q_3		
	q_3			q_4
$\leftarrow$	q_4			q_4



Estados Inacessíveis e Inúteis

- Tendo visto os algoritmos de eliminação de transições em vazio, de eliminação de não-determinismos e de eliminação de estados inacessíveis e inúteis, duas questões precisam ser respondidas:
  1. Dado um autômato finito qualquer, será possível obter uma versão determinística isenta de transições em vazio e estados inacessíveis ou inúteis, aplicando uma só vez cada um dos algoritmos apresentados?
  2. Em caso afirmativo, em qual sequência eles devem ser aplicados?

Estados Inacessíveis e Inúteis

Para responder as questões, é importante observar que:

- A eliminação de transições em vazio:
  - Pode introduzir não-determinismos.
  - Pode fazer surgir estados inacessíveis ou inúteis.
- A eliminação de não-determinismos:
  - Pode fazer surgir estados inacessíveis ou inúteis.
  - Não introduz transições em vazio.
- A eliminação de estados inacessíveis ou inúteis:
  - Não faz surgir não-determinismos.
  - Não introduz transições em vazio.

Estados Inacessíveis e Inúteis

- É fácil provar que a resposta para as questões é **sim**, bastando para isso aplicar os algoritmos na seguinte sequência:
  1. Eliminação de transições em vazio, se houver.
  2. Eliminação dos não-determinismos restantes, caso haja algum.
  3. Eliminação de estados inacessíveis ou inúteis, caso existam.
- Qualquer outra ordem poderá, dependendo do autômato, acarretar a necessidade de se aplicar um mesmo algoritmo mais de uma vez. Por exemplo:
  - A eliminação de **não-determinismos** seguida da eliminação de **transições em vazio** pode fazer com que o autômato se torne novamente não-determinístico.
  - Exigindo uma nova aplicação do algoritmo usado.

Estados Inacessíveis e Inúteis

- É fácil perceber que os AFDs isentos de transições em vazio constituem casos particulares do modelo mais geral, os **autômatos finitos não-determinísticos com transições em vazio**.
- Apesar de se tratar de um modelo simples, pode-se demonstrar que a todo AFND que apresente transições em vazio corresponde a outro AFD que aceita a mesma linguagem.
- Não-determinismos transições em vazio não contribuem em nada para aumentar o poder dos autômatos finitos quanto a uma eventual ampliação da classe de linguagens por eles reconhecidas.

Estados Inacessíveis e Inúteis

- Um importante aspecto a ser considerado no estudo dos autômatos é a:
  - possibilidade de movimentação do cursor de leitura em ambos os sentidos.
- Os autômatos finitos, efetuam a leitura de símbolos na fita de entrada em apenas um sentido (da esquerda para a direita).
- A questão que surge é: se fosse permitido aos autômatos finitos deslocar o cursor de leitura em ambos os sentidos.
  - Será que isso os tornaria mais potentes, aptos a reconhecer uma classe de linguagens mais complexa que a das linguagens regulares?

Estados Inacessíveis e Inúteis

- A resposta é **não**: é possível demonstrar que a classe de linguagens reconhecidas pelos autômatos finitos de **duplo sentido** é exatamente a mesma que é aceita pelos de sentido único, ou seja as linguagens de tipo 3.
- As linguagens regulares são muito utilizadas na especificação de linguagens de programação.
  - Os autômatos finitos representam, os modelos de implementação para tais linguagens, sendo empregados em compiladores e outras ferramentas de apoio ao desenvolvimento de programas e ao processamento de dados.

Estados Inacessíveis e Inúteis

- A profusão e a importância dos resultados teóricos existentes, referentes a esta classe de reconhecedores, torna-os atraentes em aplicações práticas.
  - Pois eles permitem a obtenção de programas muito eficientes e compactos, e ao mesmo tempo adequadamente estruturados e auto documentados.
- Devido a essas características, várias ferramentas têm sido desenvolvidas para a geração automática de programas e/ou circuitos sequenciais que realizam abstrações representadas pelos autômatos finitos, obtidos das expressões regulares.
  - Essas ferramentas vêm sendo cada vez mais utilizadas por profissionais de computação, tornando seu trabalho mais abrangente, simples, rápido e confiável.